МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
КАФЕДРА САПР

Практическая работа №1

по дисциплине

«Объектно-ориентированное программирование»

Студент гр. 5352 _____ Куренков С.О.

Преподаватель _____ Кулагин М.В.

Санкт-Петербург
2026

1. Задание

Требуется разработать консольное приложение, которое моделирует работу небольшой службы доставки.

2. Спецификация программы

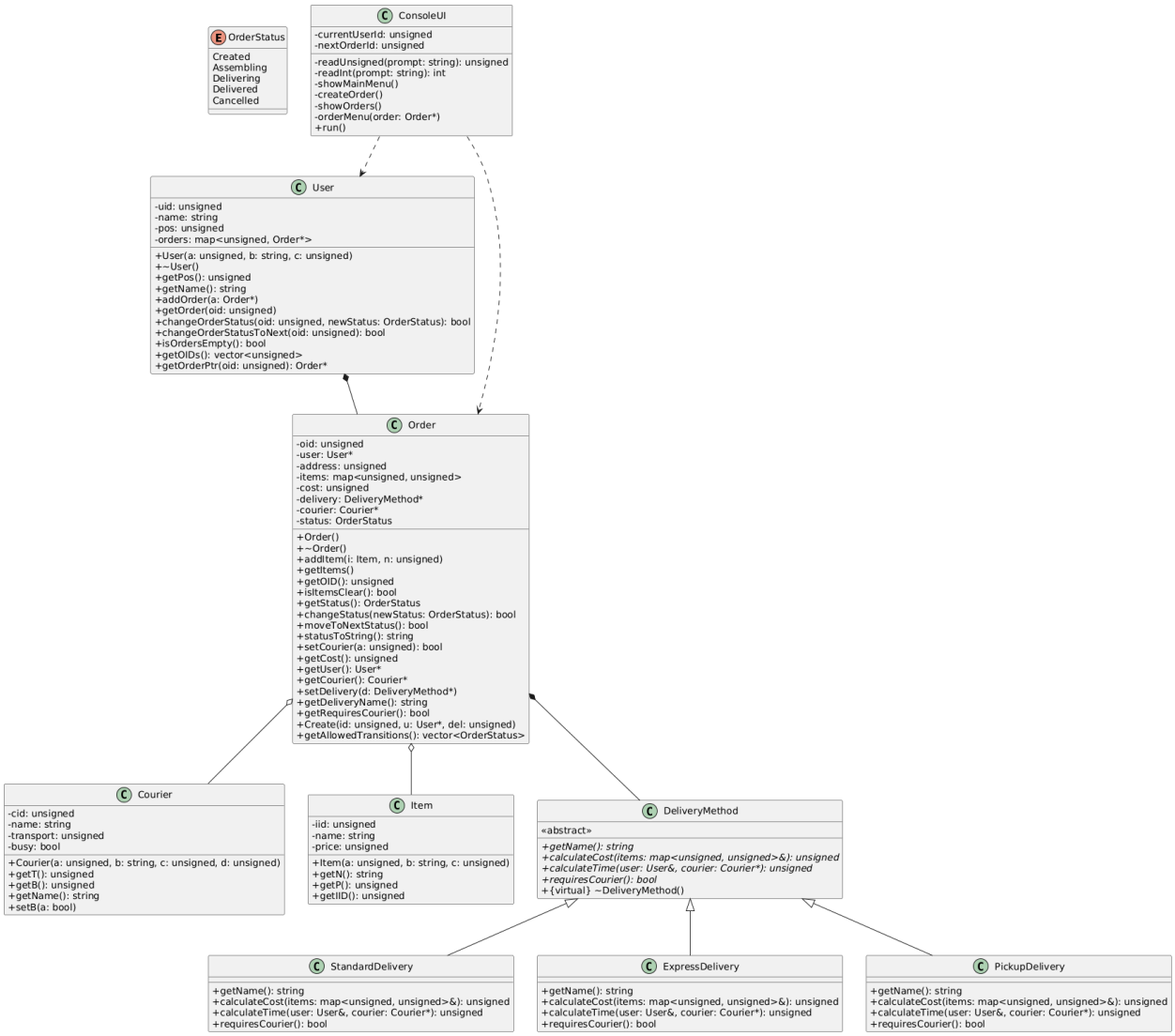


Рисунок 1 — UML-диаграмма классов.

3. Описание интерфейса пользователя программы

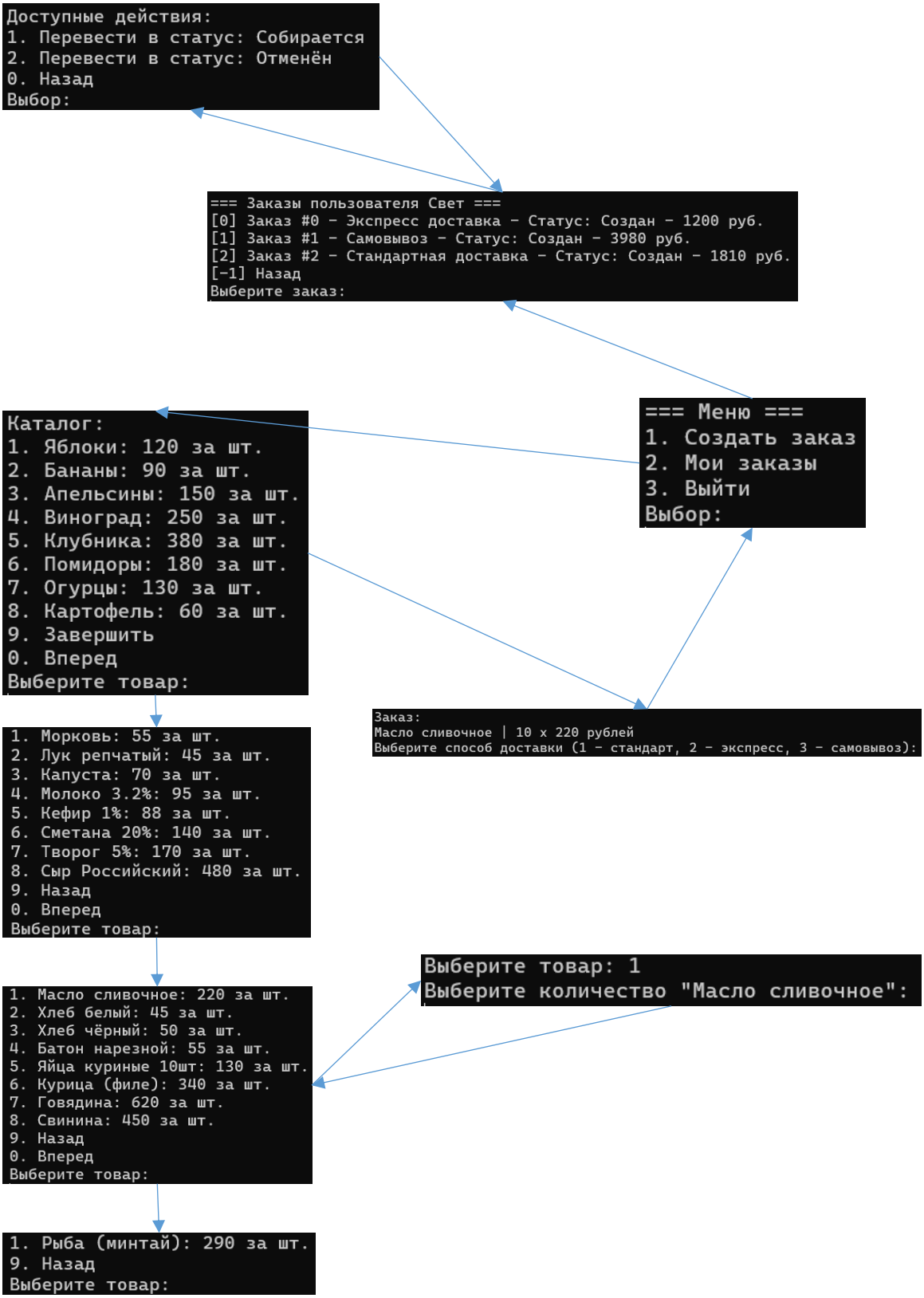


Рисунок 2 — Интерфейс программы.

4. Текст программы

```
#include <iostream>
#include <string>
#include <map>
#include <vector>
#include <algorithm>
using namespace std;

class Order;
class DeliveryMethod;

enum class OrderStatus {
		Created,
		Assembling,
		Delivering,
		Delivered,
		Cancelled
};

map<OrderStatus, vector<OrderStatus>> allowedTransitions = { // таблица переходов для этих статусов
		{ OrderStatus::Created,    { OrderStatus::Assembling, OrderStatus::Cancelled } },
		{ OrderStatus::Assembling, { OrderStatus::Delivering, OrderStatus::Cancelled } },
		{ OrderStatus::Delivering, { OrderStatus::Delivered } },
		{ OrderStatus::Delivered,  {} },
		{ OrderStatus::Cancelled,  {} }
};

string statusToStringStatic(OrderStatus s) {
		switch (s) {
		case OrderStatus::Created:    return "Создан";
		case OrderStatus::Assembling: return "Собирается";
		case OrderStatus::Delivering: return "В пути";
		case OrderStatus::Delivered:  return "Доставлен";
		case OrderStatus::Cancelled:  return "Отменён";
		}
		return "Неизвестно";
}

class User {
private:
		unsigned uid;
		string name;
		unsigned pos; // расстояние до ближайшего пункта выдачи (км)
		map<unsigned, Order*> orders; //map ссылок на заказы этого пользователя
public:
		User(unsigned a, string b, unsigned c);
		~User();

		unsigned getPos();
		string getName();
		void addOrder(Order* a);
		void getOrder(unsigned oid);
		bool changeOrderStatus(unsigned oid, OrderStatus newStatus);
		bool changeOrderStatusToNext(unsigned oid);
		bool isOrdersEmpty();
		vector<unsigned> getOIDs();
		Order* getOrderPtr(unsigned oid);
};

User::User(unsigned a, string b, unsigned c) {
		uid = a;
		name = b;
		pos = c;
}

bool User::isOrdersEmpty() {
		return orders.empty();
}

unsigned User::getPos() {
		return pos;
}

string User::getName() {
		return name;
}

class Courier {
private:
		unsigned cid;
		string name;
		unsigned transport; // 1 - пеший, 2 - вылик, 3 - машина/мотоцикл
		bool busy; // 1 - занят другим заказом, 0 - свободен
public:
		Courier(unsigned a, string b, unsigned c, unsigned d);

		unsigned getT();
```

```
		unsigned getB();
		string getName();
		void setB(bool a);
};

Courier::Courier(unsigned a, string b, unsigned c, unsigned d) {
		cid = a;
		name = b;
		transport = c;
		busy = d;
}

string Courier::getName() {
		return name;
}

unsigned Courier::getT() {
		return transport;
}

unsigned Courier::getB() {
		return busy;
}

void Courier::setB(bool a) {
		busy = a;
}

class Item {
private:
		unsigned iid; // item id
		string name;
		unsigned price;
public:
		Item(unsigned a, string b, unsigned c);

		string getN();
		unsigned getP();
		unsigned getIID();
};

Item::Item(unsigned a, string b, unsigned c) {
		iid = a;
		name = b;
		price = c;
}

string Item::getN() {
		return name;
}

unsigned Item::getP() {
		return price;
}

unsigned Item::getIID() {
		return iid;
}

// база данных
vector<User> users = { User(0, "Свет", 150), User(1, "Миша", 10), User(2, "Макс", 1000) };
vector<Courier> couriers = { Courier(0, "Арсений", 1, 0), Courier(2, "Артем", 2, 0), Courier(3, "Ярик", 3, 0) };
map<unsigned, Item> ITEMS = {
		{0,  Item(0,  "Яблоки",          120)},
		{1,  Item(1,  "Бананы",           90)},
		{2,  Item(2,  "Апельсины",       150)},
		{3,  Item(3,  "Виноград",        250)},
		{4,  Item(4,  "Клубника",        380)},
		{5,  Item(5,  "Помидоры",        180)},
		{6,  Item(6,  "Огурцы",          130)},
		{7,  Item(7,  "Картофель",        60)},
		{8,  Item(8,  "Морковь",          55)},
		{9,  Item(9,  "Лук репчатый",     45)},
		{10, Item(10, "Капуста",          70)},
		{11, Item(11, "Молоко 3.2%",      95)},
		{12, Item(12, "Кефир 1%",         88)},
		{13, Item(13, "Сметана 20%",     140)},
		{14, Item(14, "Творог 5%",       170)},
		{15, Item(15, "Сыр Российский",  480)},
		{16, Item(16, "Масло сливочное", 220)},
		{17, Item(17, "Хлеб белый",       45)},
		{18, Item(18, "Хлеб чёрный",      50)},
		{19, Item(19, "Батон нарезной",   55)},
		{20, Item(20, "Яйца куриные 10шт",130)},
		{21, Item(21, "Курица (филе)",   340)},
		{22, Item(22, "Говядина",        620)},
		{23, Item(23, "Свинина",         450)},
		{24, Item(24, "Рыба (минтай)",   290)}
};
```

```
class DeliveryMethod {
public:
	virtual ~DeliveryMethod() = default;

	virtual string getName() = 0;
	virtual unsigned calculateCost(map<unsigned, unsigned>& items) = 0;
	virtual unsigned calculateTime(User& user, Courier* courier) = 0;
	virtual bool requiresCourier() = 0;
};

class StandardDelivery : public DeliveryMethod {
public:
	string getName() override { return "Стандартная доставка"; }
	unsigned calculateCost(map<unsigned, unsigned>& items) override {
	return 300; // цена за доставку
	}
	unsigned calculateTime(User& user, Courier* courier) override {
	if (courier->getT() == 1) return user.getPos() / 5;
	if (courier->getT() == 2) return user.getPos() / 15;
	return user.getPos() / 40;
	}
	bool requiresCourier() override {
	return true;
	}
};

class ExpressDelivery : public DeliveryMethod {
public:
	string getName() override { return "Экспресс доставка"; }
	unsigned calculateCost(map<unsigned, unsigned>& items) override {
	return 450;
	}
	unsigned calculateTime(User& user, Courier* courier) override {
	if (courier->getT() == 1) return user.getPos() / 5;
	if (courier->getT() == 2) return user.getPos() / 15;
	return user.getPos() / 40;
	}
	bool requiresCourier() override {
	return true;
	}
};

class PickupDelivery : public DeliveryMethod {
public:
	string getName() override { return "Самовывоз"; }
	unsigned calculateCost(map<unsigned, unsigned>& items) override {
	return 0;
	}
	unsigned calculateTime(User& user, Courier* courier) override {
	return 0;
	}
	bool requiresCourier() override {
	return false;
	}
};

class Order {
private:
	unsigned oid;
	User* user;
	unsigned address; // тоже расстояние до ближайшего пунта выдачи
	map<unsigned, unsigned> items; // первый - item id, второй - его количество.
	unsigned cost;
	DeliveryMethod* delivery;
	Courier* courier;
	OrderStatus status;
public:
	Order();
	~Order();

	void addItem(Item i, unsigned n);
	void getItems();
	unsigned getOID();
	bool isItemsClear();
	OrderStatus getStatus();
	bool changeStatus(OrderStatus newStatus);
	bool moveToNextStatus();
	string statusToString();

	bool setCourier(unsigned a); // выбор курьера для доставки
	unsigned getCost();
	User* getUser();
	Courier* getCourier();
	void setDelivery(DeliveryMethod* d);
	string getDeliveryName();
	bool getRequiresCourier();

	void Create(unsigned id, User* u, unsigned del);
```

```
        vector<OrderStatus> getAllowedTransitions();
};

Order::Order() : delivery(nullptr), courier(nullptr), cost(0), oid(0), user(nullptr), address(0), status(OrderStatus::Created) {}

Order::~Order() {
        delete delivery;
}

vector<OrderStatus> Order::getAllowedTransitions() {
        return allowedTransitions[status];
}

bool Order::changeStatus(OrderStatus next) {
        auto& allowed = allowedTransitions[status];
        if (find(allowed.begin(), allowed.end(), next) == allowed.end()) {
         return false;
        }
        if (next == OrderStatus::Cancelled || next == OrderStatus::Delivered) {
         if (courier != nullptr) {
                courier->setB(false);
                courier = nullptr;
         }
        }
        status = next;
        return true;
}

bool Order::moveToNextStatus() {
        switch (status) {
        case OrderStatus::Created: return changeStatus(OrderStatus::Assembling);
        case OrderStatus::Assembling: return changeStatus(OrderStatus::Delivering);
        case OrderStatus::Delivering: return changeStatus(OrderStatus::Delivered);
        case OrderStatus::Delivered: return false;
        case OrderStatus::Cancelled: return false;
        }
        return false;
}

string Order::statusToString() {
        return statusToStringStatic(status);
}

User* Order::getUser() {
        return user;
}
Courier* Order::getCourier() {
        return courier;
}

bool Order::isItemsClear() {
        return items.empty();
}

void Order::setDelivery(DeliveryMethod* d) {
        delete delivery;
        delivery = d;
}

string Order::getDeliveryName() {
        return delivery->getName();
}

bool Order::getRequiresCourier() {
        return delivery->requiresCourier();
}

void Order::Create(unsigned id, User* u, unsigned del) {
        oid = id;
        user = u;
        address = user->getPos();
        courier = nullptr;

        if (delivery->requiresCourier()) {
         if (!setCourier(del)) {
                return;
         }
        }
        getCost();
}

void Order::addItem(Item i, unsigned n) {
        items[i.getIID()] += n;
}

void Order::getItems() {
        cout << "Заказ:" << endl;
        for (auto& kv : items) {
         cout << ITEMS.at(kv.first).getN() << " | " << kv.second << " x " << ITEMS.at(kv.first).getP() << " рублей" << endl;
```

```
		}
}

unsigned Order::getOID() {
		return oid;
}

bool Order::setCourier(unsigned a) {
		unsigned temp = a;
		for (unsigned j = 0; j < 2; j++)
		{
		if (temp == 1) {
				for (unsigned i = 0; i < couriers.size(); i++)
				{
						if (couriers[i].getB() == 0 and couriers[i].getT() < 3) {
								couriers[i].setB(1);
								courier = &couriers[i];
								return true;
						}
				}
		}
		for (unsigned i = 0; i < couriers.size(); i++)
		{
				if (couriers[i].getB() == 0 and couriers[i].getT() == 3) {
						couriers[i].setB(1);
						courier = &couriers[i];
						return true;
				}
		}
		temp = 1;
		}
		return false;
}

unsigned Order::getCost() {
		unsigned temp = 0;
		for (auto& kv : items) {
		temp += (kv.second * ITEMS.at(kv.first).getP());
		}

		cost = temp + delivery->calculateCost(items);
		return cost;
}

User::~User() {
		for (auto& kv : orders) {
		delete kv.second;
		}
		orders.clear();
}

void User::addOrder(Order* a) {
		if (a) {
		orders[a->getOID()] = a;
		}
}

vector<unsigned> User::getOIDs() {
		vector<unsigned> oids = {};
		for (auto& kv : orders) {
		oids.push_back(kv.first);
		}
		return oids;
}

void User::getOrder(unsigned oid) {
		cout << endl << "Заказ номер " << oid << ":" << endl;
		cout << "Заказчик: " << orders[oid]->getUser()->getName() << endl;
		if (orders[oid]->getRequiresCourier() and orders[oid]->getCourier() != nullptr) cout << "Курьер: " << orders[oid]->getCourier()->getName() << " ";
		cout << "(" << orders[oid]->getDeliveryName() << ")" << endl;
		orders[oid]->getItems();
		cout << "Итого: " << orders[oid]->getCost() << " рублей" << endl;
		cout << "Статус: " << orders[oid]->statusToString() << endl << endl;
}

bool User::changeOrderStatus(unsigned oid, OrderStatus newStatus) {
		return orders[oid]->changeStatus(newStatus);
}

bool User::changeOrderStatusToNext(unsigned oid) {
		return orders[oid]->moveToNextStatus();
}

Order* User::getOrderPtr(unsigned oid) {
		auto it = orders.find(oid);
		return (it != orders.end()) ? it->second : nullptr;
}

class ConsoleUI {
```

```
private:
	unsigned currentUserId = 0;
	unsigned nextOrderId = 0;

	unsigned readUnsigned(const string& prompt) {
	unsigned value;
	while (true) {
		cout << prompt;
		if (cin >> value) return value;
		cin.clear();
		cin.ignore(10000, '\n');
		cout << "Неверный ввод. Попробуйте снова." << endl;
	}
	}

	int readInt(const string& prompt) {
	int value;
	while (true) {
		cout << prompt;
		if (cin >> value) return value;
		cin.clear();
		cin.ignore(10000, '\n');
		cout << "Неверный ввод. Попробуйте снова." << endl;
	}
	}

	void showMainMenu() {
	cout << endl;
	cout << "=== Меню ===" << endl;
	cout << "1. Создать заказ" << endl;
	cout << "2. Мои заказы" << endl;
	cout << "3. Выйти" << endl;
	}

	void createOrder() {
	Order* myorder = new Order();

	cout << endl << "Каталог:" << endl;
	unsigned page = 0;

	unsigned startIdx; //индекс первого элемента на странице
	unsigned endIdx;   //индекс последнего элемента на странице
	unsigned choice;   //номер выбранного товара (из предоставленных в UI)
	unsigned itemIdx;  //реальный id выбранного товара
	unsigned qty = 0;      //выбранное количество товара
	while (true) {
		startIdx = page * 8;
		endIdx = min(startIdx + 8, (unsigned)ITEMS.size());

		for (unsigned i = startIdx; i < endIdx; i++) {
			cout << (i - startIdx + 1) << ". " << ITEMS.at(i).getN() << ": " << ITEMS.at(i).getP() << " за шт." << endl;
		}

		if (page > 0) cout << "9. Назад" << endl;
		else cout << "9. Завершить" << endl;
		if (endIdx < ITEMS.size()) cout << "0. Вперед" << endl;

		choice = readUnsigned("Выберите товар: ");

		if (choice == 0 && endIdx < ITEMS.size()) {
			page++;
		}
		else if (choice == 9 && page > 0) {
			page--;
		}
		else if (choice == 9) {
			break;
		}
		else if (choice >= 1 && choice <= 8 && (startIdx + choice - 1) < ITEMS.size()) {
			itemIdx = startIdx + choice - 1;

			cout << "Выберите количество \"" << ITEMS.at(itemIdx).getN() << "\": ";
			qty = readUnsigned("");

			while (qty > 10000) {
				cout << "Слишком большое количество" << endl;
				cout << "Выберите количество \"" << ITEMS.at(itemIdx).getN() << "\": ";
				qty = readUnsigned("");
			}

			if (qty != 0) myorder->addItem(ITEMS.at(itemIdx), qty);
		}
		else {
			cout << "Неверный ввод" << endl;
		}
	}

	if (myorder->isItemsClear()) {
		delete myorder;
```

```
                return;
        }

        cout << endl;
        myorder->getItems();

        unsigned del = 0;
        while (del < 1 || del > 3) {
                del = readUnsigned(
                        "Выберите способ доставки (1 - стандарт, 2 - экспресс, 3 - самовывоз): ");
        }

        if (del == 1) myorder->setDelivery(new StandardDelivery());
        else if (del == 2) myorder->setDelivery(new ExpressDelivery());
        else myorder->setDelivery(new PickupDelivery());

        myorder->Create(nextOrderId, &users[currentUserId], del);

        // Если все курьеры заняты — предложить самовывоз или отменить
        if (del != 3 && myorder->getCourier() == nullptr) {
                unsigned response = 2;
                while (response != 0 && response != 1) {
                        response = readUnsigned(
                                "Свободных курьеров нет. Выбрать самовывоз (1) или отменить заказ (0): ");
                }
                if (response == 1) {
                        myorder->setDelivery(new PickupDelivery());
                        myorder->Create(nextOrderId, &users[currentUserId], 3);
                }
                else {
                        delete myorder;
                        return;
                }
        }

        users[currentUserId].addOrder(myorder);
        nextOrderId++;

        cout << endl << "Заказ успешно создан!" << endl;
        myorder->getItems();
        cout << "Способ доставки: " << myorder->getDeliveryName() << endl;
        if (myorder->getCourier() != nullptr) {
                cout << "Курьер: " << myorder->getCourier()->getName() << endl;
        }
        cout << "Итого: " << myorder->getCost() << " рублей" << endl;
        cout << "Статус: " << myorder->statusToString() << endl;
        }

        void showOrders() {
        cout << endl;
        if (users[currentUserId].isOrdersEmpty()) {
                cout << "Здесь пока пусто :(" << endl;
                return;
        }

        vector<unsigned> OIDs = users[currentUserId].getOIDs();
        int choice;

        while (true) {
                cout << "=== Заказы пользователя " << users[currentUserId].getName() << " ===" << endl;

                for (unsigned i = 0; i < OIDs.size(); i++) {
                        Order* o = users[currentUserId].getOrderPtr(OIDs[i]);
                        cout << "[" << i << "] Заказ #" << o->getOID() << " — " << o->getDeliveryName() << " — Статус: " << o->statusToString()
<< " — " << o->getCost() << " руб." << endl;
                }
                cout << "[-1] Назад" << endl;

                choice = readInt("Выберите заказ: ");
                if (choice == -1) return;
                if (choice < 0 || choice >= (int)OIDs.size()) {
                        cout << "Неверный ввод" << endl;
                        continue;
                }

                Order* order = users[currentUserId].getOrderPtr(OIDs[choice]);
                if (order == nullptr) {
                        cout << "Заказ не найден" << endl;
                        continue;
                }
                orderMenu(order);
        }
        }

        void orderMenu(Order* order) {
        while (true) {
                cout << endl;
                users[currentUserId].getOrder(order->getOID());
```

```
                vector<OrderStatus> allowed = order->getAllowedTransitions();
                if (allowed.empty()) {
                        cout << "Заказ завершён, действия недоступны." << endl;
                        cout << "0. Назад" << endl;
                        readInt("");
                        return;
                }

                cout << "Доступные действия:" << endl;
                for (size_t i = 0; i < allowed.size(); i++) {
                        cout << (i + 1) << ". Перевести в статус: "
                                << statusToStringStatic(allowed[i]) << endl;
                }
                cout << "0. Назад" << endl;

                int choice = readInt("Выбор: ");
                if (choice == 0) return;
                if (choice < 1 || choice >(int)allowed.size()) {
                        cout << "Неверный ввод" << endl;
                        continue;
                }

                if (!order->changeStatus(allowed[choice - 1])) {
                        cout << "Нельзя перевести заказ в этот статус" << endl;
                }
                else {
                        cout << "Новый статус: " << order->statusToString() << endl;
                }
        }
        }

        public:
        void run() {
                while (true) {
                        showMainMenu();
                        unsigned choice = readUnsigned("Выбор: ");
                        switch (choice) {
                        case 1: createOrder(); break;
                        case 2: showOrders();  break;
                        case 3: return;
                        default: cout << "Неверный ввод" << endl;
                        }
                }
        }
};

int main() {
        setlocale(LC_ALL, "Russian");

        ConsoleUI ui;
   ui.run();

   return 0;
}
```

5. Выводы

Было разработано консольное приложение, которое моделирует работу небольшой службы доставки, результаты были выложены на Github: https://github.com/OchenKrutoyPelmen/OOP-lab1